a(16) = 113 for OEIS A389241: a witness and an exhaustive search re-executed independently

Frédéric David Blum
Catalyst AI Research, Haifa, Israel
ORCID 0009-0009-2487-2974 · freddavidblum@catalystais.com
26 October 2026 · version 2

Abstract. Sequence A389241 of the OEIS gives, for each n, the maximum number of distinct sums of consecutive terms of a permutation of 1..n; it was known up to a(15) = 100. We show a(16) = 113. A permutation of 1..16 with 113 distinct consecutive sums gives the lower bound. An exhaustive branch-and-bound search over all permutations of 1..16, split into 60 runs that together cover the 240 possible ordered first pairs, finds none with 114 or more. Each run first recovers the known values a(3)..a(9) and checks the branch-and-bound against brute force; each run was re-executed by a different AI agent with a byte-identical output. The work was organised and verified by Maï, a multi-agent research system developed by Catalyst AI Research. The exact programs, their outputs, SHA-256 fingerprints and a checker that verifies all records in seconds are published with this record, and any run can be re-executed.

1. The sequence

For a permutation $p = (p_1, ..., p_n)$ of 1..n, consider the n(n+1)/2 sums of consecutive terms $p_i + ... + p_j$, $1 \le i \le j \le n$, and let s(p) be the number of distinct values among them. A389241(n) is the maximum of s(p) over all permutations of 1..n. The known terms end with a(15) = 100 [1]. For n = 16 there are 136 consecutive sums, so a(16) ≤ 136; this note shows a(16) = 113.

2. Lower bound: a(16) ≥ 113

The permutation

```
[1, 2, 6, 11, 13, 4, 14, 9, 12, 8, 15, 16, 10, 5, 7, 3]
```

has exactly 113 distinct consecutive sums. This takes a few lines to check (`verify.py` in [2]); a second program written independently (`recount.py`) gives the same count. The permutation was found on 20 October 2026 by the branch-and-bound program of Section 3, run on the prefix [1, 2].

3. Upper bound: no permutation of 1..16 reaches 114

Algorithm. The search is a depth-first branch-and-bound written in C (compiled with `gcc -O2`) and embedded in a short Python driver. A permutation is built from left to right. When a value x is appended, the new consecutive sums are the sums ending at x; the program counts how many of them repeat a value already present. Appending terms never removes a sum, so a repetition, once it occurs, persists in every completion of the prefix. If D repetitions have occurred, every completion has at most 136 − D distinct sums; the branch is abandoned as soon as 136 − D ≤ 113. A branch therefore survives only if it could still exceed 113, and a completed permutation is reported only if it has at least 114 distinct sums. The bound is exact bookkeeping, not a heuristic: no permutation reaching 114 can be pruned.

Partition of the search. Every permutation of 1..16 starts with one of the 240 ordered pairs (a, b), a ≠ b. The program takes a list of prefixes (its only variable line, `PREFIXES`) and explores all permutations starting with each of them. The search was split into runs of four pairs each. The 60 runs published here cover all 240 pairs (some pairs are covered by more than one run); the checker reads the covered prefixes from the programs themselves, not from metadata.

Positive control. Before the search, every run computes a(3)..a(9) by brute force over all permutations, prints them (6, 9, 13, 19, 25, 32, 39, the known values) and checks that the branch-and-bound returns

the same maxima as brute force for n = 3..9, for every first element and for every first pair. A run whose control fails does not count. This guards against a program that reports nothing because it searches nothing.

Result. For each of the 240 pairs, the output reports `trouve_geq114 False`: no permutation of 1..16 with that prefix has 114 or more distinct consecutive sums. With Section 2, **a(16) = 113**.

4. Records, independent re-execution and checking

Every run was executed on a dedicated compute host without network access that records the SHA-256 of the exact program and of its standard output. Every run was then re-executed, from the same program, by a different AI agent, and the two outputs have the same SHA-256 (byte-identical). The output contains no timing or random value, so identity is expected from a correct run. The 60 runs took about 270 CPU hours and the re-executions about 293 more.

Hosts. Until 26 October 2026 the re-executions ran on the same compute host as the original runs (*host-1*); from that date they run on a second, independent host (*host-2*). In this record 2 of the 60 re-executions ran on host-2. An earlier version of the repository described the first phase-3 re-execution as made "on a different server": it was submitted by a different agent but ran on the same host; the repository carries a dated correction.

Checking. `phase3/verify_phase3.py` [2] checks, for each run: the SHA-256 of the published program; that the program is the reference search up to its PREFIXES line, comments and blank lines; the SHA-256 of the output and its equality with the re-execution; the positive control; and, for each prefix, that the best score is at most 113 and that no permutation reaches 114. It then checks that the prefixes cover all 240 pairs. Any run can be re-executed with `python3 phase3/codes/lotNN.py` (1 to 6 CPU hours each; gcc required).

5. How the computation was organised: Maï

The work was organised and carried out by **Maï**, a multi-agent research system developed by Catalyst AI Research. In Maï, AI agents running on separate servers split a research mission into tasks, launch the computations on dedicated compute hosts that record every program and every output, and check each other's work. For a(16), three Maï agents (based on Claude Haiku, Anthropic), each on its own server, split the search, launched the runs and checked each other's results. The organising principle is that no agent can make a result count by stating it: the computations are executed and recorded by the compute host, a result is accepted only after re-execution by another agent with an identical output, and every program must first recover known values. The model cost of the agents for the exhaustive search was about 14 US dollars. The method by which Maï's agents coordinate is the subject of patent applications and is not described here; the mathematical result does not depend on it, since every computation is published and can be re-run.

6. Limitations

The upper bound rests on one program (one algorithm, one implementation). Re-executions check that the runs were performed and reported faithfully, not that the program is correct; that rests on the simplicity of the bound (Section 3), on the positive controls, and on anyone's ability to read and re-run the code. A second, independent implementation, or a formal proof of the pruning rule, would strengthen the result further. Most re-executions ran on the same host as the originals (Section 4).

Data availability

The witness, the 60 exact programs, their outputs and fingerprints, the reference program and the checkers are in the repository [2] and archived with this record.

References

[1] OEIS Foundation Inc., The On-Line Encyclopedia of Integer Sequences, A389241, https://oeis.org/A389241.

[2] F. D. Blum, mailleproof: witness and exhaustive-search certificate for A389241(16), https://github.com/davidangularme/mailleproof (2026).

Appendix: the 60 runs

Prefixes searched by each run (read from its program), duration of the original run and of its independent re-execution, and the host of the re-execution. Durations in hours.

Run	Prefixes (first two elements)	Original	Re-exec.	Host
1	[1,2], [1,3], [1,4], [1,5], [1,2], [1,3], [1,4], [1,5]	4.3	3.9	host-1
2	[1,6], [1,7], [1,8], [1,9], [1,6], [1,7], [1,8], [1,9]	4.0	3.7	host-1
3	[1,10], [1,11], [1,12], [1,13], [1,10], [1,11], [1,12], [1,13]	3.6	4.6	host-1
4	[1,14], [1,15], [1,16], [2,1]	3.7	5.9	host-1
5	[2,3], [2,4], [2,5], [2,6]	4.1	3.8	host-1
6	[2,7], [2,8], [2,9], [2,10], [2,7], [2,8], [2,9], [2,10]	6.0	5.1	host-2
7	[2,11], [2,12], [2,13], [2,14]	3.3	5.4	host-1
8	[2,15], [2,16], [3,1], [3,2]	3.6	6.0	host-1
9	[3,4], [3,5], [3,6], [3,7], [3,4], [3,5], [3,6], [3,7]	4.9	5.8	host-1
10	[3,8], [3,9], [3,10], [3,11], [3,8], [3,9], [3,10], [3,11]	3.3	5.9	host-1
11	[3,12], [3,13], [3,14], [3,15], [3,12], [3,13], [3,14], [3,15]	3.7	5.5	host-1
12	[3,16], [4,1], [4,2], [4,3]	4.9	6.0	host-1
13	[4,5], [4,6], [4,7], [4,8]	3.8	6.1	host-1
14	[4,9], [4,10], [4,11], [4,12]	3.9	5.4	host-1
15	[4,13], [4,14], [4,15], [4,16]	5.6	5.6	host-1
16	[5,1], [5,2], [5,3], [5,4], [5,1], [5,2], [5,3], [5,4]	4.2	6.9	host-1
17	[5,6], [5,7], [5,8], [5,9]	4.3	5.5	host-1
18	[5,10], [5,11], [5,12], [5,13]	5.3	5.3	host-1
19	[5,14], [5,15], [5,16], [6,1]	4.4	5.6	host-1
20	[6,2], [6,3], [6,4], [6,5]	4.6	5.8	host-1
21	[6,7], [6,8], [6,9], [6,10], [6,7], [6,8], [6,9], [6,10]	5.1	5.5	host-1
22	[6,11], [6,12], [6,13], [6,14]	5.7	5.7	host-1
23	[6,15], [6,16], [7,1], [7,2]	4.6	5.5	host-1
24	[7,3], [7,4], [7,5], [7,6]	4.6	5.3	host-1
25	[7,8], [7,9], [7,10], [7,11]	5.2	4.4	host-2
26	[7,12], [7,13], [7,14], [7,15]	4.3	4.9	host-1
27	[7,16], [8,1], [8,2], [8,3]	5.2	5.7	host-1
28	[8,4], [8,5], [8,6], [8,7]	4.4	5.2	host-1
29	[8,9], [8,10], [8,11], [8,12]	5.4	5.7	host-1
30	[8,13], [8,14], [8,15], [8,16], [8,13], [8,14], [8,15], [8,16]	4.8	4.9	host-1
31	[9,1], [9,2], [9,3], [9,4], [9,1], [9,2], [9,3], [9,4]	4.6	5.3	host-1
32	[9,5], [9,6], [9,7], [9,8]	5.0	5.1	host-1
33	[9,10], [9,11], [9,12], [9,13]	4.9	4.8	host-1
34	[9,14], [9,15], [9,16], [10,1]	4.7	4.7	host-1
35	[10,2], [10,3], [10,4], [10,5], [10,2], [10,3], [10,4], [10,5]	5.1	4.9	host-1
36	[10,6], [10,7], [10,8], [10,9]	5.1	5.0	host-1
37	[10,11], [10,12], [10,13], [10,14]	4.8	5.0	host-1
38	[10,15], [10,16], [11,1], [11,2]	4.9	4.5	host-1
39	[11,3], [11,4], [11,5], [11,6]	4.8	4.8	host-1
40	[11,7], [11,8], [11,9], [11,10]	4.8	4.5	host-1
41	[11,12], [11,13], [11,14], [11,15], [11,12], [11,13], [11,14], [11,15]	4.5	4.2	host-1
42	[11,16], [12,1], [12,2], [12,3]	4.6	4.6	host-1
43	[12,4], [12,5], [12,6], [12,7]	4.7	4.7	host-1
44	[12,8], [12,9], [12,10], [12,11]	4.4	4.8	host-1
45	[12,13], [12,14], [12,15], [12,16]	4.1	3.8	host-1
46	[13,1], [13,2], [13,3], [13,4], [13,1], [13,2], [13,3], [13,4]	4.6	4.5	host-1
47	[13,5], [13,6], [13,7], [13,8], [13,5], [13,6], [13,7], [13,8]	4.8	4.5	host-1
48	[13,9], [13,10], [13,11], [13,12]	4.3	3.8	host-1
49	[13,14], [13,15], [13,16], [14,1]	4.6	4.0	host-1
50	[14,2], [14,3], [14,4], [14,5], [14,2], [14,3], [14,4], [14,5]	4.6	4.4	host-1
51	[14,6], [14,7], [14,8], [14,9]	4.5	4.3	host-1

Run	Prefixes (first two elements)	Original	Re-exec.	Host
52	[14,10], [14,11], [14,12], [14,13]	4.1	4.0	host-1
53	[14,15], [14,16], [15,1], [15,2], [14,15], [14,16], [15,1], [15,2]	4.0	4.2	host-1
54	[15,3], [15,4], [15,5], [15,6], [15,3], [15,4], [15,5], [15,6]	4.3	3.9	host-1
55	[15,7], [15,8], [15,9], [15,10]	4.2	4.6	host-1
56	[15,11], [15,12], [15,13], [15,14], [15,11], [15,12], [15,13], [15,14]	3.9	3.9	host-1
57	[15,16], [16,1], [16,2], [16,3]	4.2	3.6	host-1
58	[16,4], [16,5], [16,6], [16,7]	4.6	4.0	host-1
59	[16,8], [16,9], [16,10], [16,11]	3.9	4.1	host-1
60	[16,12], [16,13], [16,14], [16,15]	3.7	3.2	host-1